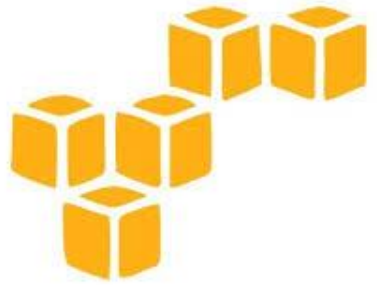
Infrastructure as Code

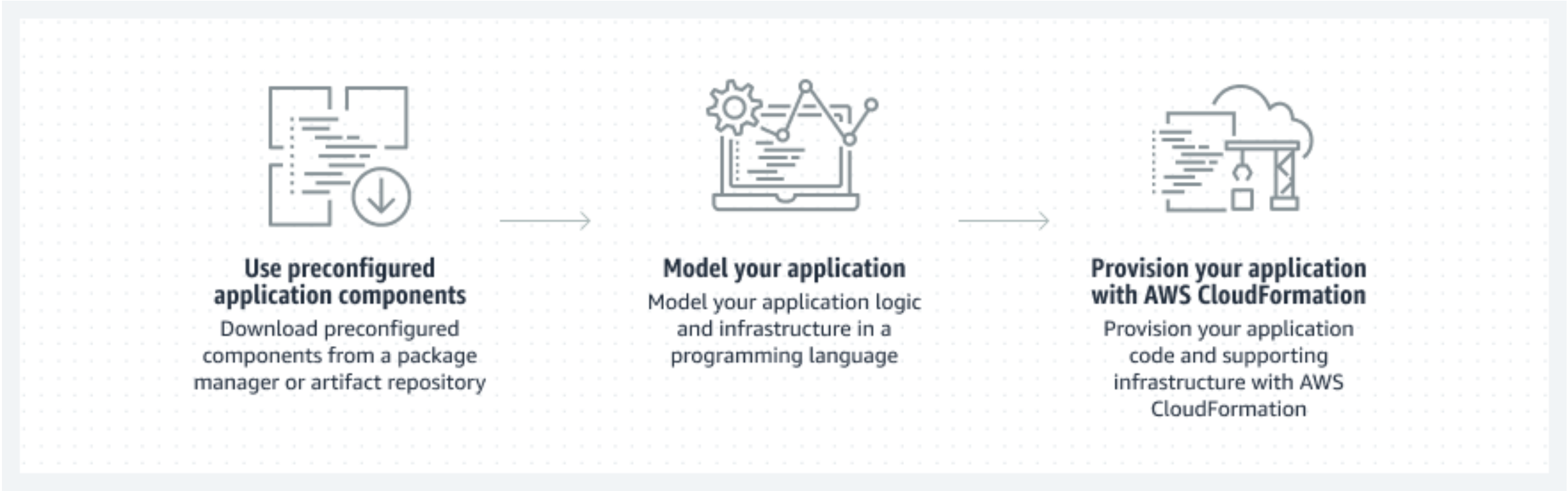
Cloud Development Kit

Ron C. Chiang
University of St. Thomas



AWS CDK

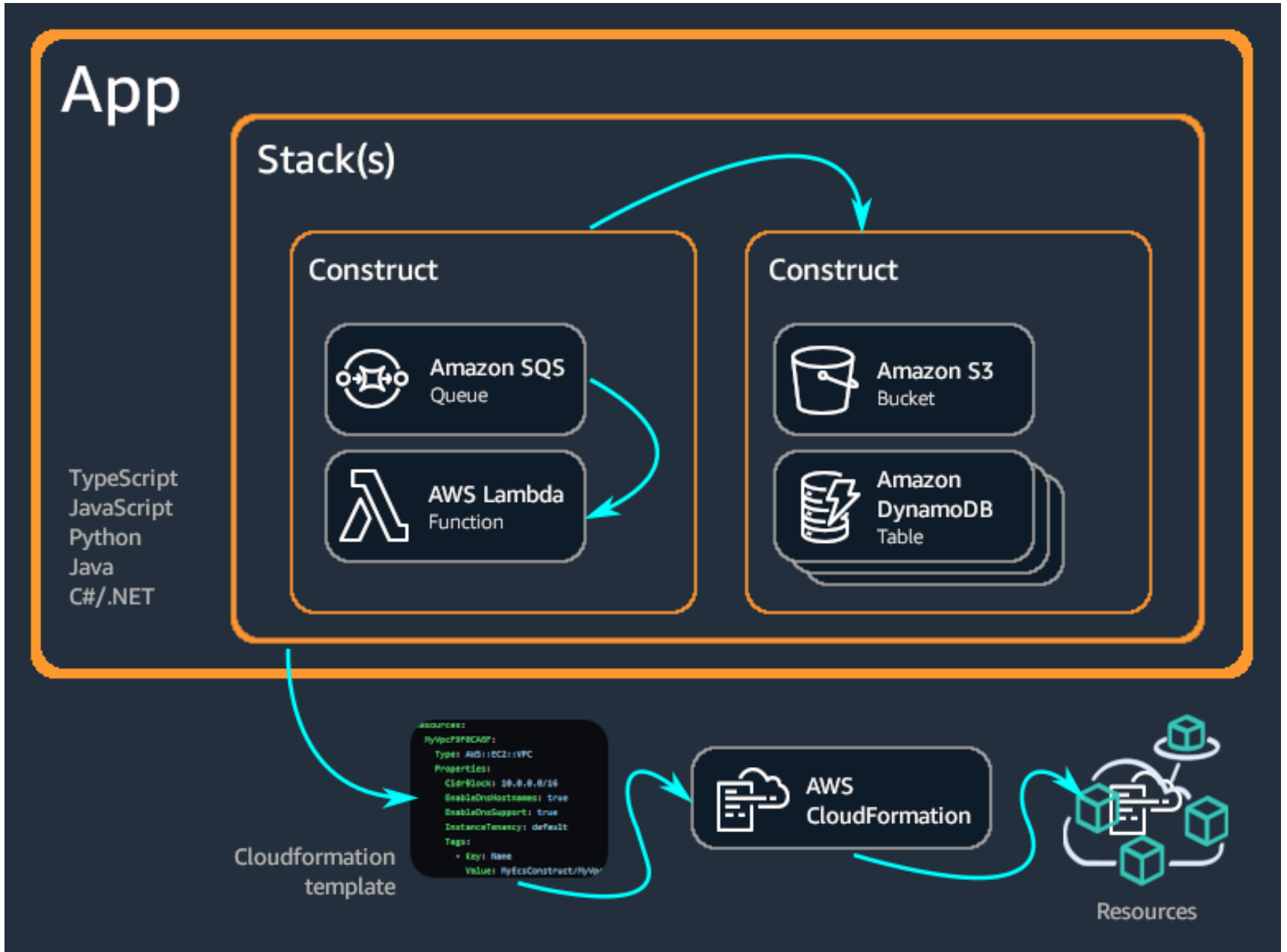
- A framework for defining cloud infrastructure in code and provisioning it through AWS CloudFormation



Benefits of using CDK

- Facilitate the building of reliable, scalable, cost-effective applications in the cloud with the considerable expressive power of a programming language
- Developers can use one of many supported programming languages (such as TypeScript, JavaScript, Python, Java, C#/.Net, and Go) to define reusable cloud components known as ***Constructs***. You compose these together into ***Stacks*** and ***Apps***

Constructs, Stacks, and Apps



Prerequisites

- AWS Account and User
- AWS CLI
- Node.js (14.15 or later) is always required to use CDK, but we can build App using other supported programming languages
- IDE for your programming language
  - We will use AWS Cloud9 because it satisfies almost all prerequisites
- AWS CDK Toolkit

Prepare the working environment - Cloud9

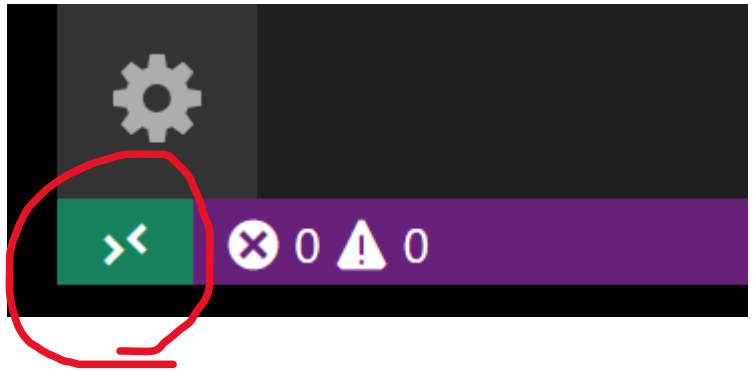
- Open an environment in AWS Cloud9
- Ensure the latest security updates and bug fixes are installed
  - `sudo yum –y update`
- Install Node Version Manager (nvm), for installing Node.js later
  - To check/confirm
    - `nvm --version`
  - If not found, to install
    - https://github.com/nvm-sh/nvm#installing-and-updating

Prepare the working environment – VS Code

- Cloud9 stop accepting new customers after July 2024
- Alternative:
  - VS Code + remote servers (e.g., EC2 instances, Azure VMs)
  - VS Code + Dev Containers
- Download and Install VS Code on your workstation. Then, install Remote-SSH Extension in VS Code
- Launch an EC2 instance
  - AMI: Amazon Linux 2023. Use Default VPC. SG: Open port 22 to your IP
  - Make sure you have the key for SSH

Connecting to a remote server in VS Code

- In VS Code, click on the bottom-left corner to open SSH extension.
- Select "connect to host". Then, "configure SSH hosts"
- Open and edit the configure file ~/.ssh/config
- Config example ==>




```
Host cdk-lab
    HostName ec2-44-212-43-134.compute-1.amazonaws.com
    User ec2-user
    IdentityFile ~\Downloads\iac-fall-2024.pem
```

Prepare the Remote Dev Env for VS Code

- Connect to the remote server
- Install the Devlopment Environment
  - sudo yum groupinstall "Development Tools"
- Ensure the latest security updates and bug fixes are installed
  - `sudo yum -y update`
- Install Node Version Manager (nvm), for installing Node.js later
  - To check/confirm
    - `nvm --version`
  - If not found, to install
    - https://github.com/nvm-sh/nvm#installing-and-updating

Prepare the working environment (cont.)

- Install Node.js
  - Check/Confirm
    - `node --version`
  - If not found, to install
    - `nvm install node`
- Install the AWS CDK
  - Check/Confirm
    - `cdk --version`
  - If not found, to install
    - `npm install -g aws-cdk`



Prepare the working environment (cont.)

- Bootstrapping
  - Deploying stacks with the AWS CDK requires dedicated Amazon S3 buckets and other containers to be available to AWS CloudFormation during deployment. To bootstrap, issue:
    - `cdk bootstrap aws://ACCOUNT-NUMBER/REGION`



Development Workflow

1. Create the app from a template provided by the AWS CDK.
2. Add code to the app to create resources within stacks.
3. (Optional) Build the app. (The AWS CDK Toolkit does this for you if you forget.)
   - catches syntax and type errors
4. Synthesize one or more stacks in the app to create an AWS CloudFormation template.
   - catches logical errors in defining your AWS resources
5. Deploy one or more stacks to your AWS account.
   - may find permission issues

Your First AWS CDK App



Create the app

- Create a project directory. Then, make it the current directory.
  - `mkdir hello-cdk`
  - `cd hello-cdk`

- Initialize the app.
  - `cdk init app --language python`

- Activate the app's Python virtual environment and install the AWS CDK core dependencies.
  - `source .venv/bin/activate`
  - `python -m pip install -r requirements.txt`

Build and List the stacks

- Build the app
  - In most programming environments, after changing your code, you build (compile) it.
  - This isn't strictly necessary with the AWS CDK—the Toolkit does it for you so that you can't forget. But you can still build manually whenever you want to catch syntax and type errors.
- List the stacks in the app, to verify that everything is working correctly.
  - `cdk ls`
  - If you don't see HelloCdkStack, make sure you named your app's directory hello-cdk.

Add an Amazon S3 bucket

- Edit hello_cdk/hello_cdk_stack.py

```
import aws_cdk as cdk
import aws_cdk.aws_s3 as s3

class HelloCdkStack(cdk.Stack):

    def __init__(self, scope: cdk.App, construct_id: str, **kwargs) -> None:
        super().__init__(scope, construct_id, **kwargs)

        bucket = s3.Bucket(self, "MyFirstBucket", versioned=True)
```

Synthesize



- Synthesize an AWS CloudFormation template for the app
  - cdk synth

- If your app contained more than one stack, you'd need to specify which stack or stacks to synthesize. But since it only contains one, the CDK Toolkit knows you must mean that one.

Deploy

- To deploy the stack using AWS CloudFormation
  - cdk deploy

- As with cdk synth, you don't need to specify the name of the stack since there's only one in the app

- Congratulations! You've deployed your first stack using AWS CDK



Updating your app

- If you modify your app, the AWS CDK can update your deployed resources. Let’s try it!
- Change the bucket so it can be automatically deleted when deleting the stack
  - changing the bucket's RemovalPolicy
  - use the autoDeleteObjects property to ask the AWS CDK to delete the objects from the bucket before destroying it
- Note: AWS CloudFormation doesn't delete S3 buckets that contain any objects.

Update hello_cdk/hello_cdk_stack.py.

```
bucket = s3.Bucket(self, "MyFirstBucket",
    versioned=True,
    removal_policy=cdk.RemovalPolicy.DESTROY,
    auto_delete_objects=True)
```

- Use cdk diff to see changes (next page)
- Use cdk deploy to update
  - It will warn you about the changes. Enter y to approve.

diff outputs

- **IAM Statement Changes** and **IAM Policy Changes** - These permission changes are there because we set the AutoDeleteObjects property on our Amazon S3 bucket. The auto-delete feature uses a custom resource to delete the objects in the bucket before the bucket itself is deleted. The IAM objects grant the custom resource's code access to the bucket.

- **Parameters** - The AWS CDK uses these entries to locate the Lambda function asset for the custom resource.

- **Resources** - The new and changed resources in this stack. We can see the previously mentioned IAM objects, the custom resource, and its associated Lambda function being added. We can also see that the bucket's DeletionPolicy and UpdateReplacePolicy attributes are being updated. These allow the bucket to be deleted along with the stack, and to be replaced with a new one.

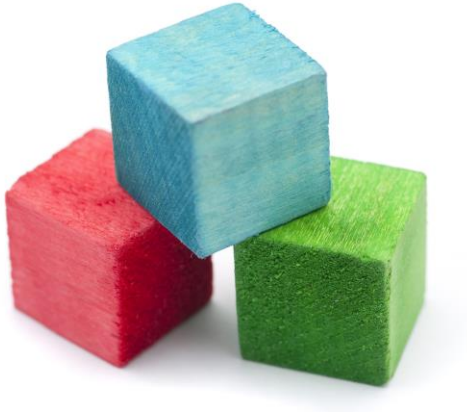
Clean up

- Now that you're done with your first app, destroy your app's resources to avoid incurring any costs from the bucket you created
  - `cdk destroy`

- Enter y to approve the changes and delete any stack resources

- NOTE: If we didn't change the bucket's RemovalPolicy, the stack deletion would complete successfully, but the bucket would become orphaned (no longer associated with the stack).

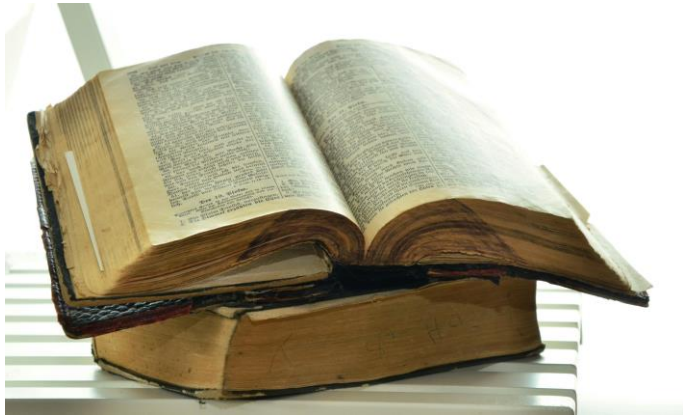
Constructs



- The basic building blocks of AWS CDK apps.
- A construct represents a "cloud component“
- Encapsulates everything AWS CloudFormation needs to create the component.

Construct Library and References

- The AWS CDK includes a collection of constructs called the AWS Construct Library, containing constructs for every AWS service.
- For example, the s3.Bucket class represents an Amazon S3 bucket, and the dynamodb.Table class represents an Amazon DynamoDB table.
- API references
- Construct Hub is a resource to help you discover additional constructs from AWS, third parties, and the open-source CDK community.



Construct Levels

- A (*lower-level*) construct can represent a single AWS resource, such as an Amazon Simple Storage Service (Amazon S3) bucket.

- A construct can also be a *higher-level* abstraction consisting of multiple related AWS resources. Examples
  - a worker queue with its associated compute capacity
  - a scheduled job with monitoring resources and a dashboard.

L1 Constructs

- Low-level constructs, or called *CFN* resources
- Directly represent all resources available in AWS CloudFormation
- For example, CfnBucket represents the AWS::S3::Bucket AWS CloudFormation resource.
- Must explicitly configure all resource properties. In other words, a complete understanding of the details of the underlying AWS CloudFormation resource model.

L2 Constructs

- Also represent AWS resources, but with a higher-level, intent-based API
  - similar functionality, but incorporate the defaults, boilerplate, and glue logic you'd be writing yourself with a CFN Resource construct
- Offer convenient *defaults* and reduce the need to know all the details about the AWS resources they represent
- Also provide convenience *methods* that make it simpler to work with the resource
  - such as bucket.addLifeCycleRule(), which adds a lifecycle rule to the bucket

L3 Constructs

- Also called *patterns*
- Designed to help you complete common tasks in AWS, often involving multiple kinds of resources
- For example,
  - aws-ecs-patterns.ApplicationLoadBalancedFargateService construct represents an architecture that includes an AWS Fargate container cluster employing an Application Load Balancer
  - aws-apigateway.LambdaRestApi construct represents an Amazon API Gateway API that's backed by an AWS Lambda function

Composition

- The key pattern for defining higher-level abstractions through constructs
- A high-level construct can be composed from any number of lower-level constructs
- Composition lets you define *reusable* components and share them like any other code

Initialization

- Constructs are implemented in classes that extend the Construct base class.
- You define a construct by instantiating the class.
- All constructs take three parameters when they are initialized.
  - *scope*, *id*, and *props*
- For example,
  - bucket = s3.Bucket(self, "MyFirstBucket", versioned=True)

scope id props

scope

- The construct's parent or owner, either a stack or another construct, which determines its place in the construct tree.

- You should usually pass *this* (or *self* in Python), which represents the current object, for the scope.

- bucket = s3.Bucket(self, "MyFirstBucket", versioned=True)

id

- An identifier that must be unique within this scope.
- The identifier serves as a namespace for everything that's defined within the current construct.
- It's used to generate unique identifiers such as resource names and AWS CloudFormation logical IDs.
- bucket = s3.Bucket(self, "MyFirstBucket", versioned=True)

props

- A set of properties or keyword arguments, depending upon the language, that define the construct's initial configuration.

- In most cases, constructs provide sensible defaults, and if all props elements are optional, you can omit the props parameter completely

- bucket = s3.Bucket(self, "MyFirstBucket", versioned=True)

Apps and Stacks

- Your CDK application is called an *app,* which is represented by the AWS CDK class App.

- Take our current working project as an example, the following codes from app.py instantiate an app with a single stack.

```
app = cdk.App()
HelloCdkStack(app, "HelloCdkStack")
```

Interacting with Constructs

- After you instantiate a construct, the construct object exposes a set of methods and properties that let you interact with the construct and pass it around as a reference to other parts of the system.

- Most AWS constructs have a set of grant methods that you can use to grant AWS Identity and Access Management (IAM) permissions on that construct to a principal.

- The following example grants the IAM group data-science permission to read from the Amazon S3 bucket raw-data.

```
raw_data = s3.Bucket(self, 'raw-data')
data_science = iam.Group(self, 'data-science')
raw_data.grant_read(data_science)
```

Interacting with Constructs – another example

- Another common pattern is for AWS constructs to set one of the resource's attributes from data supplied elsewhere. Attributes can include Amazon Resource Names (ARNs), names, or URLs.

- The following code defines an AWS Lambda function and associates it with an Amazon Simple Queue Service (Amazon SQS) queue through the queue's URL in an environment variable.

```
jobs_queue = sqs.Queue(self, "jobs")
create_job_lambda = lambda_.Function(self, "create-job",
      runtime=lambda_.Runtime.NODEJS_18_X,
      handler="index.handler",
      code=lambda_.Code.from_asset("./create-job-lambda-code"),
      environment=dict(
          QUEUE_URL=jobs_queue.queue_url
      )
)
```



The Construct Tree

- In AWS CDK apps, you define constructs "inside" other constructs using the scope argument passed to every construct. In this way, an AWS CDK app defines a hierarchy of constructs known as *the construct tree*.
- The *root* of this tree is your *app*—that is, an instance of the App class.
- Within the app, you instantiate one or more *stacks*.
- Within stacks, you instantiate either AWS CloudFormation resources or higher-level *constructs*, which may themselves instantiate resources or other constructs, and so on down the tree.

Traversing the tree

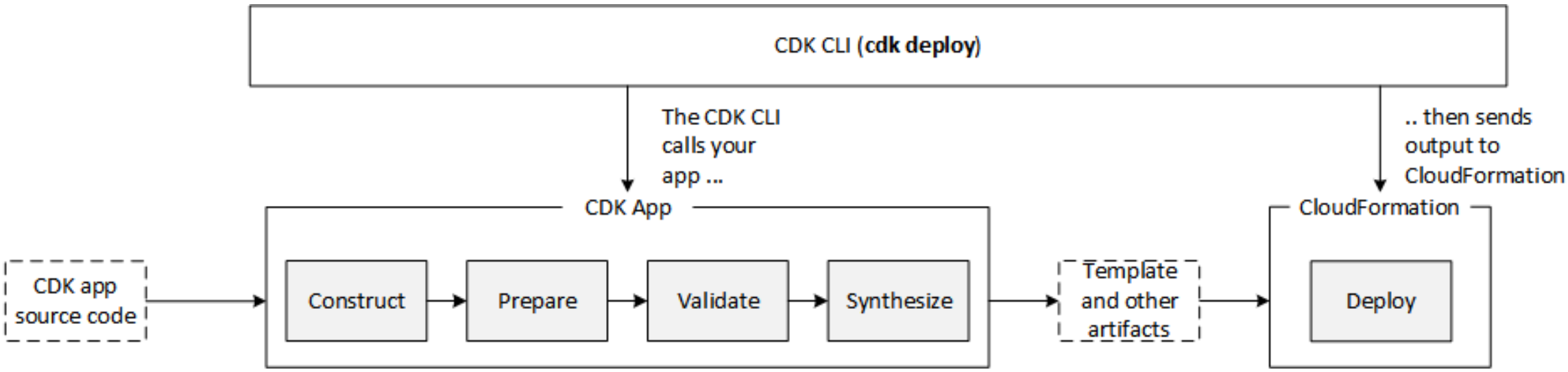
- The AWS CDK uses the IDs of all constructs in the path from the tree's root to each child construct to generate the unique IDs required by AWS CloudFormation.
- This approach means that construct IDs only need to be unique within their scope, rather than within the entire stack as in native AWS CloudFormation.
  - However, if you move a construct to a different scope, its generated stack-unique ID changes, and AWS CloudFormation won't consider it the same resource.

- The construct tree is separate from the constructs that you define in your AWS CDK code. However, it's accessible through any construct's node attribute, which is a reference to the node that represents that construct in the tree.
- Each node is a Node instance, the attributes of which provide access to the tree's root and to the node's parent scopes and children.

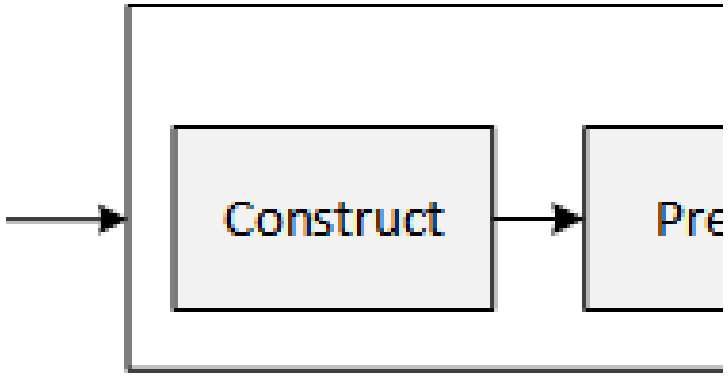
Traversing the tree (cont.)

- node.children – The direct children of the construct.
- node.id – The identifier of the construct within its scope.
- node.path – The full path of the construct including the IDs of all of its parents.
- node.root – The root of the construct tree (the app).
- node.scope – The scope (parent) of the construct, or undefined if the node is the root.
- node.scopes – All parents of the construct, up to the root.
- node.uniqueId – The unique alphanumeric identifier for this construct within the tree (by default, generated from node.path and a hash).

App Lifecycle

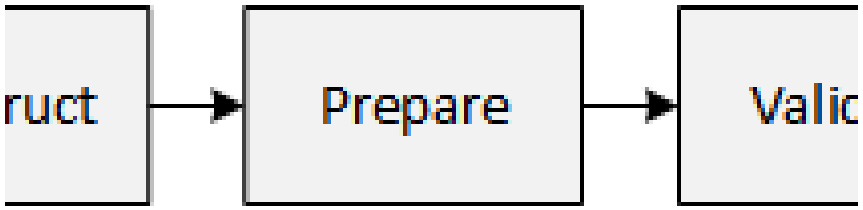


Construction (initialization) phase



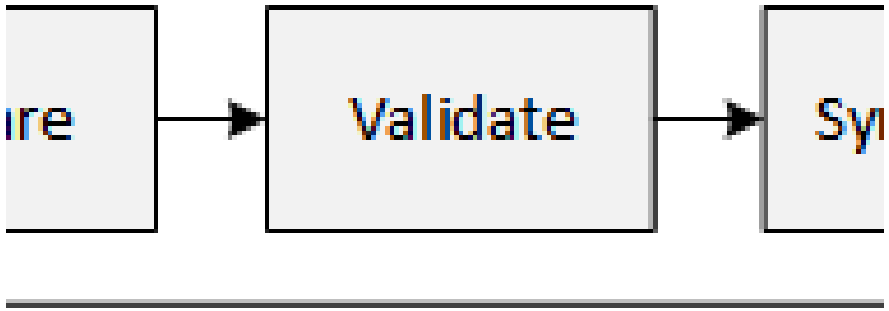
- Your code instantiates all of the defined constructs and then links them together.
- In this stage, all of the constructs (app, stacks, and their child constructs) are instantiated and the constructor chain is executed.
- Most of your app code is executed in this stage

Preparation phase



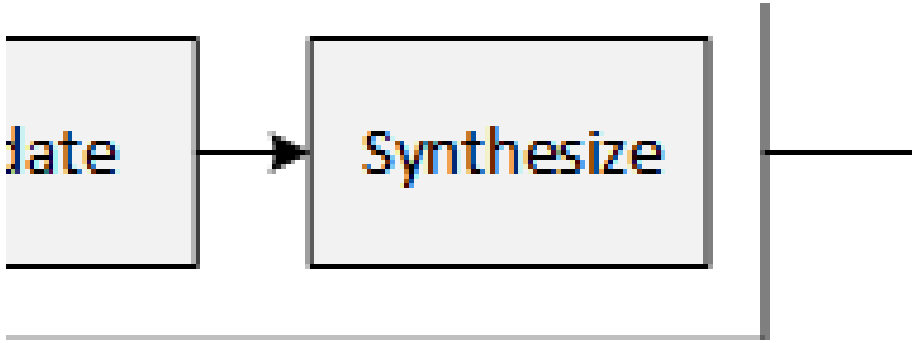
- All constructs that have implemented the prepare method participate in a final round of modifications, to set up their final state.
- The preparation phase happens automatically. As a user, you don't see any feedback from this phase. It's rare to need to use the "prepare" hook, and generally not recommended.
- Be very careful when mutating the construct tree during this phase, because the order of operations could impact behavior.

Validation phase



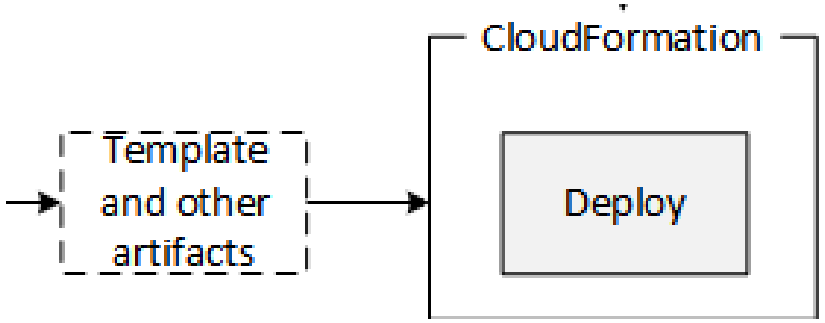
- All constructs that have implemented the validate method can validate themselves to ensure that they're in a state that will correctly deploy.
- You will get notified of any validation failures that happen during this phase.
- Generally, we recommend performing validation as soon as possible (usually as soon as you get some input) and throwing exceptions as early as possible.
- Performing validation early improves diagnosability as stack traces will be more accurate, and ensures that your code can continue to execute safely.

Synthesis phase



- This is the final stage of the execution of your AWS CDK app.
- It's triggered by a call to app.synth(), and it traverses the construct tree and invokes the synthesize method on all constructs.
- Constructs that implement synthesize can participate in synthesis and emit deployment artifacts to the resulting cloud assembly.
- These artifacts include AWS CloudFormation templates, AWS Lambda application bundles, file and Docker image assets, and other deployment artifacts.
- Cloud assemblies describes the output of this phase. In most cases, you won't need to implement the synthesize method.

Deployment phase



- In this phase, the AWS CDK Toolkit takes the deployment artifacts cloud assembly produced by the synthesis phase and deploys it to an AWS environment.
- It uploads assets to Amazon S3 and Amazon ECR, or wherever they need to go.
- Then, it starts an AWS CloudFormation deployment to deploy the application and create the resources.

AWS Resource Attributes

- Most resources in the AWS Construct Library expose attributes, which are resolved at deployment time by AWS CloudFormation.
- Attributes are exposed in the form of properties on the resource classes with the type name as a prefix.
- The following example shows how to get the URL of an Amazon SQS queue using the queue_url property.

```
import aws_cdk.aws_sqs as sqs

queue = sqs.Queue(self, "MyQueue")
url = queue.queue_url # => A string representing a deploy-time value
```

Referencing resources

- CDK classes require properties that are resource objects.
  - For example, an Amazon ECS resource requires a reference to the cluster on which it runs.
  - Another example, an Amazon CloudFront distribution requires a reference to the bucket containing the source code.
- To refer to a resource
  - passing a resource defined in your CDK app, either in the same stack or in a different one
  - passing a proxy object referencing a resource defined in your AWS account, created from a unique identifier of the resource (such as an ARN)

Referencing resources - example

- The following example defines an Amazon ECS cluster and then uses it to define an Amazon ECS service.

```
cluster = ecs.Cluster(self, "Cluster")

service = ecs.Ec2Service(self, "Service", cluster=cluster)
```

Static Website on S3

- We have created a bucket using CDK. Let's use it to host a static website.
- First, update the bucket settings to allow public read.

```
block_public_access=s3.BlockPublicAccess.BLOCK_ACLS_ONLY,
access_control=s3.BucketAccessControl.BUCKET_OWNER_FULL_CONTROL,
public_read_access = True,
```

- Then, enable static website hosting.

```
website_index_document = "index.html",
```

- Wait a second, where is the index.html?
- Let's deploy the content to the bucket!

Deploy web content

```
import aws_cdk as cdk
import aws_cdk.aws_s3 as s3
import aws_cdk.aws_s3_deployment as s3deployment

class HelloCdkStack(cdk.Stack):

    def __init__(self, scope: cdk.App, construct_id: str, **kwargs) -> None:
        super().__init__(scope, construct_id, **kwargs)

        bucket = s3.Bucket(self, "MyFirstBucket",
                           block_public_access=s3.BlockPublicAccess.BLOCK_ACLS_ONLY,
                           access_control=s3.BucketAccessControl.BUCKET_OWNER_FULL_CONTROL,
                           public_read_access=True,
                           website_index_document="index.html",
                           versioned=True,
                           removal_policy=cdk.RemovalPolicy.DESTROY,
                           auto_delete_objects=True)

        deployment = s3deployment.BucketDeployment(self, "DeployWebsite",
                             sources=[s3deployment.Source.asset('./hello_cdk/static-website.zip')],
                             destination_bucket=bucket)
```